

COST-COGNIZANT COMBINATORIAL TEST CASE PRIORITIZATION[#]

ZIYUAN WANG*, LIN CHEN[†], BAOWEN XU^{‡,¶} and YAN HUANG[§]

*State Key Laboratory for Novel Software Technology
Nanjing University, Department of Computer Science and Technology
Nanjing University, Nanjing, Jiangsu, 210093, China*

*wangziyuan@nju.edu.cn

[†]lchen@nju.edu.cn

[‡]bwxu@nju.edu.cn

[§]yanhuang@nju.edu.cn

Combinatorial testing has been widely used in practice. People usually assume all test cases in combinatorial test suite will run completely. However, in many scenarios where combinatorial testing is needed, for example the regression testing, the entire combinatorial test suite is not run completely as a result of test resource constraints. To improve the efficiency of testing, combinatorial test case prioritization technique is required. For the scenario of regression testing, this paper proposes a new cost-cognizant combinatorial test case prioritization technique, which takes both combination weights and test costs into account. Here we propose a series of metrics with physical meaning, which assess the combinatorial coverage efficiency of test suite, to guide the prioritization of combinatorial test cases. And two heuristic test case prioritization algorithms, which are based on total and additional techniques respectively, are utilized in our technique. Simulation experimental results illustrate some properties and advantages of proposed technique.

Keywords: Software testing; combinatorial testing; test case prioritization; metric; algorithm.

1. Introduction

Software could be considered as a complex logic system, which may be affected by many parameters or factors, such as system configurations, internal events, external inputs etc. and each parameter may have many different available options. An intuitive and simple software testing approach for such kind of software is named as Each Choice, which requires each option or value of each parameter to be included in at least one test case [2]. However, it is reported from numerous case studies that many faults in software may be involved in the combinations of options

[#]A preliminary version [1] of parts of this paper was presented at the 22nd International Conference on Software Engineering and Knowledge Engineering (SEKE2010).

[¶]Correspondence could also be addressed to Prof. Baowen Xu besides the first author Dr. Ziyuan Wang.

from different parameters [3]. This phenomenon shows us that, rather than the single factors, the interaction of multiple parameters may also affect the work of software systems.

When testing such interactions, combinatorial testing (or interaction testing in some references) is considered as a practical software testing technique, since it uses a small test suite that cover all needed parametric values and their combinations, to detect the faults that may triggered by these single parameters in software and even the interactions of them. For the test object (or system under test) that has n parameters, it may be unacceptable to cover all possible n -tuple combinations of parametric options or values, since the combinatorial explosion may lead a large number of test cases in exhaustive test suite, especially when n is a large integer. However, in such a scenario, combinatorial testing could provide a tradeoff between the cost of software testing and the degree of combinatorial coverage. For example, 2-way combinatorial testing (or pair-wise testing) requires covering all the 2-tuple combinations of parametric values rather than the n -tuples.

For instance, considering the configurations of an imaginary network application, in which there are 4 configuration parameters, including browser, operation system, internet connection, and database. Each configuration parameter has 3 available options, which are shown in Table 1. Exhaustive testing, which covers all the possible 4-tuple combinations, requires $3^4 = 81$ test cases. Meanwhile pair-wise testing requires only 9 test cases to cover all pairs of parametric values, which are shown in Table 2.

Table 1. Configurations of imaginary network application.

Browser	OS	Connection	DB
Firefox	Linux	LAN	DB/2
IE	Windows	ISDN	Oracle
Opera	Macintosh	Modem	Access

Table 2. Pair-wise test suite for imaginary network application.

No.	Browser	OS	Connection	DB
1	Firefox	Linux	LAN	DB/2
2	Firefox	Windows	ISDN	Oracle
3	Firefox	Macintosh	Modem	Access
4	IE	Linux	ISDN	Access
5	IE	Windows	Modem	DB/2
6	IE	Macintosh	LAN	Oracle
7	Opera	Linux	Modem	Oracle
8	Opera	Windows	LAN	Access
9	Opera	Macintosh	ISDN	DB/2

Combinatorial testing has been studied and applied widely for its efficiency and effectiveness. Many applications of combinatorial testing have indicated that a carefully designed combinatorial test suite which contains small number of test cases can yield high fault detection ability with reduced test cost. Especially, it is effective for some highly configurable systems [4]. In most circumstances that combinatorial testing is needed, pairwise testing has been already considered as a sufficient approach rather than the higher way combinatorial testing.

Generally, when using combinatorial testing approach, all test cases in combinatorial test suite are considered as fair ones and they should be run completely to achieve the given combinatorial coverage criteria. However, in many scenarios where combinatorial testing is suitable to use, for example regression testing, the entire combinatorial test suite could not run completely as a result of test resource constraints. To improve the efficiency of regression testing, people usually adopt test case prioritization technique between successive releases of a software object. To prioritize combinatorial test cases in regression testing, the combinatorial test case prioritization technique is needed to extend traditional combinatorial testing technique. There may be numerous different kinds of information could help us to prioritize combinatorial test cases. Here firstly we consider the importance or the weight of combinations of parametric values. In intuition, test cases should be prioritized to cover more important or weighty combinations of parametric values as soon as possible. It means that if testing process is terminated exceptionally, more important parametric values and their combinations could be tested already. And it is also necessary to take the cost of test cases into account. Also in intuition, test cases that require less cost should be run earlier to save the testing resource. It means that if testing process is terminated exceptionally, more test cases could be run since their less cost. Without loss of generality, we could assume the “cost” of a test case means the consumed time when running this test case.

In this paper, we propose a novel cost-cognizant combinatorial test case prioritization technique. This technique takes both the importance of combinations and the costs of test cases into consideration. The first specific contributions of our work is proposing a series of novel physical meaningful metrics for the cost-cognizant combinatorial test case prioritization, which takes both the importance of combinations and costs of testing into account, to evaluate the average rate of combinatorial coverage of executed test cases. A well defined metric is very essential for test case prioritization problem, since the final achievement of all possible prioritization techniques are to increase the metric value of prioritized test suite. After defining metrics, we adopt two heuristic algorithms, which are respectively based on the well-known total and additional techniques in the field of test case prioritization, to prioritize or sort test cases according to the proposed metrics. Finally, we design simulation experiments, and demonstrate some properties of proposed technique and its advantage over existing methods.

This paper is organized as follows. Section 2 presents some backgrounds, including combinatorial testing and test case prioritization. Section 3 proposes the

problem of cost-cognizant combinatorial test case prioritization, and deduces a series of corresponding metrics. Section 4 describes the test case prioritization algorithms. Section 5 provides simulation experimental results. Finally, related works, conclusion, and future works are given successively.

2. Preliminaries

In the following section, some preliminaries about covering array, combinatorial testing and test case prioritization techniques are presented.

2.1. Covering array and combinatorial testing

Combinatorial testing aims to cover some specific combinations of parametric values by using a small number of test cases in combinatorial test suite. Generally, combinatorial test suite is designed based on the covering array, which is a special kind of mathematical structure that extends the original notion of orthogonal array.

To introduce the notation of covering array that is related to combinatorial testing, we should model the test object in combinatorial testing. Considering a test object that has n parameters, we suppose each parameter f_i has a_i parametric values ($i = 1, 2, \dots, n$). Without loss of generality, we use $F = \{f_1, f_2, \dots, f_n\}$ to denote the set of these n parameters, and $V_i = \{1, 2, \dots, a_i\}$ to denote the set of valid values for the parameter f_i ($i = 1, 2, \dots, n$). Here we could define $a = \max_{1 \leq i \leq n} \{a_i\}$. If the cardinalities of all these value sets are uniform ($a = a_1 = a_2 = \dots = a_n$), we say it is a system with fixed-level factors, and these parameters could be also denoted as $F = \{a^n\}$. Otherwise, it is a system with mixed-level factors.

For the fixed-level systems, we have orthogonal array and fixed-level covering array.

Definition 1. An orthogonal array $OA_\lambda(m; \tau, n, a)$ is an $m \times n$ array on totally a symbols with the property that each $m \times \tau$ sub-array contains all ordered subsets from a symbols of size τ exactly λ times.

Definition 2. A τ -way fixed-level covering array $CA(m; \tau, n, a)$ is an $m \times n$ array on totally a symbols with the property that each $m \times \tau$ sub-array contains all ordered subsets from a symbols of size τ at least once.

In earlier period, people usually use orthogonal array to design combinatorial test suite, which has the advantage of regression analysis on testing results. However, there may be also some disadvantages. Besides the difficult construction of orthogonal array, some τ -tuple combinations may be covered redundantly if $\lambda > 1$. Also note that in many situations, orthogonal array exists only for a great integer λ . Therefore, fixed-level covering array, which could be generated more easily, may be more suitable for software testing, since it does not require the redundant combinatorial coverage.

Similarly, for the fixed-level systems, we have mixed-level covering array.

Definition 3. A τ -way mixed-level covering array $CA(m; \tau, (a_1, a_2, \dots, a_n))$ is an $m \times n$ array on totally a symbols, where $a = \max_{1 \leq i \leq n} \{a_i\}$, the j th column ($1 \leq j \leq n$) contains only the elements from the set V_j of size a_j , and each $m \times \tau$ sub-array contains all τ -tuple combinations of values from the τ columns at least once.

Both τ -way fixed-level covering array and τ -way mixed-level covering array could be called τ -way covering array. We can conclude from the definition of τ -way covering array that, all the τ -tuple combinations of parametric values in the following set $CombSet$ must be covered by the τ -way covering array at least once:

$$CombSet = \bigcup_{r \in R} CombSet_r \quad (1)$$

where

$$R = \{\{f_{i,1}, f_{i,2}, \dots, f_{i,\tau}\} \mid f_{i,1}, f_{i,2}, \dots, f_{i,\tau} \in F\} \subseteq 2^F \quad (2)$$

$$CombSet_r = \{(v_{i,1}, v_{i,2}, \dots, v_{i,\tau}) \mid r = \{f_{i,1}, f_{i,2}, \dots, f_{i,\tau}\}, \\ v_{i,1} \in V_{i,1}, v_{i,2} \in V_{i,2}, \dots, v_{i,\tau} \in V_{i,\tau}\}. \quad (3)$$

We found that the set R contains all C_n^τ possible τ -cardinality subsets from n parameters in F . Since all these subsets contain the same number (which is exactly τ) of parameters, we say τ -way covering array as a fixed-strength covering array, where the fixed-strength is τ . Similarly, we can define mixed-strength or variable-strength covering array also, by assuming the cardinality of different parameter subsets in R are mixed or variable. So R could be any subset of 2^F , where for any two parameter subsets $r_i, r_j \in R$, we have $r_i \not\subseteq r_j$ and $r_j \not\subseteq r_i$ [5]. Note that variable-strength covering array could also be classified as fixed-level one and mixed-level one.

Definition 4. A variable-strength covering array $VCA(m; (a_1, a_2, \dots, a_n), R)$ is an $m \times n$ array on all a symbols, where $a = \max_{1 \leq i \leq n} \{a_i\}$, the j th column ($1 \leq j \leq n$) contains only the elements from the set V_j of size a_j , and each combination in the following $CombSet$ is contained at least once:

$$CombSet = \bigcup_{r \in R} CombSet_r \quad (4)$$

where

$$R = \{\{f_{i,1}, f_{i,2}, \dots, f_{i,\tau_j}\} \mid f_{i,1}, f_{i,2}, \dots, f_{i,\tau_j} \in F\} \subseteq 2^F \quad (5)$$

$$CombSet_r = \{(v_{i,1}, v_{i,2}, \dots, v_{i,\tau_j}) \mid r = \{f_{i,1}, f_{i,2}, \dots, f_{i,\tau_j}\}, \\ v_{i,1} \in V_{i,1}, v_{i,2} \in V_{i,2}, \dots, v_{i,\tau_j} \in V_{i,\tau_j}\}. \quad (6)$$

For a given test object, a τ -way combinatorial test suite could be obtained easily from a τ -way fixed-level or mixed-level covering array. Similarly, we can easily obtain a variable-strength combinatorial test suite from a variable strength fixed-level or

mixed-level covering array. Here, each row of covering array, which is an n -tuple $(v_1, v_2, \dots, v_n)$ where $v_1 \in V_1, v_2 \in V_2, \dots, v_n \in V_n$, is equivalent to a test case of combinatorial test suite for the test object. Therefore, we assume that combinatorial test suite and covering array are equivalent in this paper.

The software testing approach, which designs and runs a τ -way covering array or variable-strength covering array as a test suite, is the τ -way combinatorial testing or variable-strength combinatorial testing.

To reduce the cost of testing, one of the most key challenges in combinatorial testing is to generate the optimal test suite with minimal number of test cases. But unfortunately, it is proven that the problem of generating combinatorial test suite is an NP-C problem [6]. To solve this problem, people develop many combinatorial test generation methods. Most of these methods can be categorized into the following classes: algebraic direct construction, algebraic recursive construction, greedy heuristic methods, meta-heuristic search methods, and others including integer programming, constraint solving, etc. [7].

2.2. Test case prioritization

Test case prioritization technique seeks to schedule test cases in an order that increase the effectiveness at meeting some given performance goal. This problem is defined as:

Definition 5. For an initial non-prioritized test suite T_{init} , the test case prioritization aims to find a $T \in PT$ such that:

$$(\forall T')(T' \in PT)(T \neq T')[f(T) \geq f(T')] \quad (7)$$

in which PT is the set of permutations of all test cases in T_{init} and f is an objective function that, applied to any such ordering, yields an award value for that ordering [8].

Although there have been many possible objective functions to guide the prioritizing or sorting test cases, most of people restrict their attentions to techniques that would improve test suite's efficiency of faults detection, since the fault detection is considered as the final mission of software testing. To formally illustrate how rapidly a prioritized test suite detects faults, a well-known objective function, which is named as *average percentage of faults detection* or *APFD* metric for short, was proposed [9]. Considering a test suite $T = \{T_1, T_2, \dots, T_m\}$, and a prophetic fault set $FaultSet = \{fault_1, fault_2, \dots, fault_k\}$ of test object. If tf_i is denoted as the index of the first test case that detects the $fault_i$, the *APFD* for T is:

$$APFD(T) = 1 - \frac{tf_1 + tf_2 + \dots + tf_k}{k \times m} + \frac{1}{2m}. \quad (8)$$

A prioritized test suite with higher *APFD* value has better fault detection rates than those with lower values. There are some mutation versions of *APFD* metric, including the *APFD_C* that accounts for both varying test costs and fault

severities [10], the *NAPFD* that accounts both fault detection and the number of executed test cases [11], etc.

In order to achieve the goal that improving test suite's *APFD*, *APFD_C*, or *NAPFD*, an intuitive method is prioritizing test cases according to the relationship between test suite *T* and detectable fault set *FaultSet*. However, unfortunately it is impossible to get such a relationship accurately before running test cases. Therefore, some substitutive test case prioritization techniques, which use some other types of information to define objective functions, are more practical.

For different scenarios, many test case prioritization techniques, which use different types of information, have been proposed. E.g., there are techniques based on source code and test history [9, 12], varying fault severities and test costs [10], basic block coverage [13], time budget [14], program slicing [15], etc. In order to prioritize test cases for these scenarios, several algorithms have been proposed. Most of them could be divided into two classes: greedy methods and meta-heuristic search methods.

3. Cost-Cognizant Combinatorial Test Case Prioritization

In the scenarios where the weights of combinations and costs of test cases are taken into consideration in combinatorial testing, although the final mission of testing is still to detect faults rapidly, the most available information that we could use in combinatorial test case prioritization evidently includes the combination weights and test costs. In this section, we describe the problem of cost-cognizant combinatorial test case prioritization detailedly. Especially, a series of metrics, which are objective functions for this problem, are deduced to evaluate combinatorial test suite's efficiency of combinatorial coverage.

3.1. Definitions

Some related definitions about cost-cognizant combinatorial test case prioritization are given in this sub-section.

As we mentioned previously, combinatorial testing, which provides a sample from the exhaustive test suite, aims to cover some specific combinations of parametric values. Especially, both the fixed-strength and variable-strength combinatorial testing aims to covers all the combinations in a specific set *CombSet*, which has been described in Formula 1 and Formula 4 respectively in previous contexts. Because of such a uniform model for both the fixed-strength and variable-strength combinatorial testing, we can, with loss of generality, only focus on the test prioritization problem for the fixed-strength τ -way combinatorial test suite in this paper.

In traditional combinatorial testing process, we assumes that all test cases in a combinatorial test suite are fair, which means that they have uniform priorities. So they will be run fairly one by one, until the entire test suite has been executed completely. In such a testing process, the running order of different test cases

should be random. However, there may be some circumstances that testers think that some specific τ -tuple combinations of parametric values are more important than others. For example, these combinations may be involved with some special parametric values, including the values that are related to the modified blocks, preferred choices, important functions, etc. Therefore, to improve the efficiency of combinatorial testing, those test cases that include such important combinations should be run earlier.

Here we can use a positive value, which could be also called the weight for the combination of parametric values, to evaluate the importance of such combination. It means that, for each combination $c_k \in CombSet$, we denote the weight of the combination c_k as $Weight(c_k) = cw_k$ ($k = 1, 2, \dots, |CombSet|$). Therefore, the weighted benefit of a given test case $test$ could be considered as the summation of weights of each combination that covered by such test case:

$$Benefit(test) = \sum_{c_k \in tCov(test)} cw_k \tag{9}$$

in which the function $tCov(test)$ returns a set of all τ -tuple combinations that are covered by the current test case $test$. Then the l -biased covering array can be defined as follows [16].

Definition 6. An l -biased covering array is a covering array, where the total weighted benefit of first l test cases is as large as possible.

It is known that finding l -biased covering array is an NP-Hard problem [16]. So people usually use the term **biased covering array** to mean the covering array in which the test cases have been ordered, and the first l test cases could yield a large total weighted benefit for each integer l [16]. Here we can easily get a prioritized τ -way combinatorial test suite from a τ -way biased covering array.

Besides the weights of combinations, some other types of information, such as the costs of test cases, could also be taken into consideration when prioritizing test cases. However, up to date, the costs of running test cases have not been considered in previous combinatorial test case prioritization techniques yet. So the costs of all combinatorial test cases are assumed to be a uniform value. However, in real applications, in order to save the testing resource, the test cases that need less cost should be run earlier if all test cases have the same conditions in all the other aspects. Here the cost of test case may include consumed time, power, computer occupied, etc. Without loss of generality, in this paper, we assume the “cost of test case” means the consumed time when running this test case. For a test case $T_i \in T$, we can denote the cost of T_i as $Cost(T_i) = tc_i$ ($i = 1, 2, \dots, |T|$). If the cost of test case is taken into consideration in traditional combinatorial testing, testers may require a **cost-cognizant covering array**, which will be defined following, to design test suite. It is evidently easy to get a cost-cognizant covering array by sorting all test cases according to their costs in ascending order. The detail is omitted here.

Definition 7. A cost-cognizant covering array is a covering array, where the total cost of first l test cases is as less as possible for any integer l .

Then if both the weights of combinations and the costs of test case are taken into consideration, it is evident that testers want to obtain more weighted benefits, which was described in Formula 1, with less cost in testing. So we have:

Definition 8. A cost-cognizant l -biased covering array is a covering array, where the ratio of total weighted benefit of first l test cases to their total cost is as large as possible.

It is evident that finding a cost-cognizant l -biased covering array is not easier than an NP-Hard problem, since the problem of finding l -biased covering array has already been proven to be an NP-Hard problem. Similarly, we could use the term **cost-cognizant biased covering array**, in which the test cases have been ordered, and the first l test cases could yield a large ratio of total weighted benefit to the total cost for any l . Therefore, in the circumstance that both the weights of combinations and costs of test cases are taken into consideration, a prioritized τ -way combinational test suite is equivalent to a τ -way cost-cognizant biased covering array. We need to develop cost-cognizant combinatorial test case prioritization techniques to find the cost-cognizant biased covering array.

3.2. Proposed metrics

In the following sub-section, we propose a series of metrics, which could evaluate the performance of a given cost-cognizant biased covering array. They are also considered as the objective functions to guide the finding of prioritized test suite. Here we restrict our attention to the improvement of prioritized combinatorial test suite's efficiency of obtaining benefits, which has been defined previously. Since the benefit is dependent on the τ -tuple combination coverage ability, to formally illustrate how rapidly a test suite obtains benefits, we will use the term **average percentage of combinatorial coverage** or *APCC* for short, as the name of the series metrics in this paper.

3.2.1. Metric without combination weights and test costs

Firstly, we discuss the traditional scenario where both combination weights and test costs are ignored. Considering a prioritized τ -way combinational test suite $T = \{T_1, T_2, \dots, T_m\}$, we need a step function that exactly illustrate the improvement of combinatorial coverage ability in testing process. The input of such a function should be a real number that ranges from 0 to m , which means the number of executed test cases. It outputs the number of covered τ -tuple combinations at different periods of the testing process.

After the running of the i th test case T_i , all executed test cases include exactly $T_1, T_2, \dots$, and T_i during the period when the $i + 1$ th test case T_{i+1} is still running.

So all the τ -tuple combinations that have been covered by executed test cases could form a set:

$$AddtlCov_i = \bigcup_{j=1}^i tCov(T_j), \quad i = 1, 2, \dots, m \tag{10}$$

in which, the function $tCov(T_j)$ returns a set of all τ -tuple combinations covered by the test case T_j . Then the step function $fun(x)$ could be defined as:

$$fun(x) = |AddtlCov_{\lfloor x \rfloor}|, \quad 0 \leq x \leq m. \tag{11}$$

This function can exactly illustrate the additional improvement of the number of covered τ -tuple combinations. We take a simple test object with $F = \{2^3\}$ where there are three parameters and each parameter has two parametric values, and corresponding pair-wise test suite that shown in Table 3 as examples. In the case that both combination weights and test costs are ignored, we ignore the columns about weighted benefit and cost in Table 3. The step function $fun(x)$ for a given order of test cases in Table 3 is shown in Fig. 1(a).

After defining such a step function for $T = \{T_1, T_2, \dots, T_m\}$, we can immediately find that: (i) when all tests run completed, there must be $fun(m) = |AddtlCov_m| = |CombSet|$; and (ii) for any real number $x < m$, the larger $fun(x)$ leads the more rapidly coverage of τ -tuple combinations. To evaluate how rapidly a prioritized test

Table 3. Pair-wise test suite for test object with $F = \{2^3\}$.

Test No.	f_1	f_2	f_3	Number of covered 2-tuple	Weighted benefit	Cost
1.	1	1	1	3	3.00	4.0
2.	1	2	2	3	1.25	3.0
3.	2	1	2	3	1.25	2.0
4.	2	2	1	3	1.25	1.0

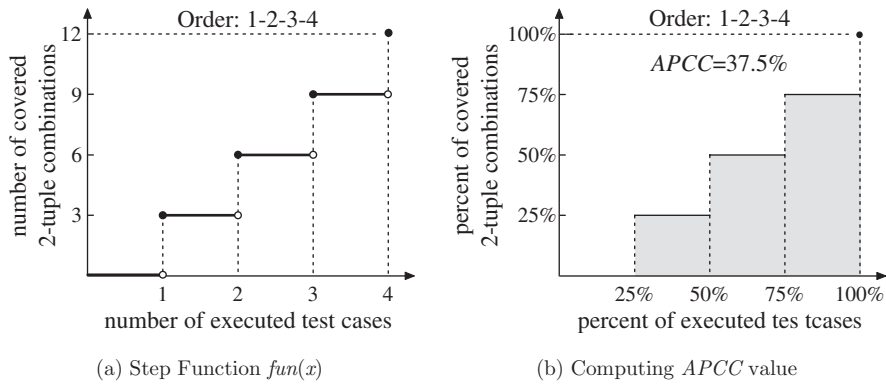


Fig. 1. Step function and $APCC$ metric for prioritized test cases.

suite covers combinations in the whole testing process, we compute a definite integral for $fun(x)$ to get a temporary metric:

$$Temp_Metric(T) = \int_0^m fun(x) = \sum_{i=1}^{m-1} \left| \bigcup_{j=1}^i tCov(T_j) \right|. \quad (12)$$

Then, by normalizing the above, we get the $APCC$ metric for the traditional scenario of combinatorial test case prioritization, where both combination weights and test costs are ignored. It is denoted as:

$$APCC(T) = \frac{\sum_{i=1}^{m-1} \left| \bigcup_{j=1}^i tCov(T_j) \right|}{m \times |CombSet|}. \quad (13)$$

For test cases shown in Table 3 and the given order 1-2-3-4, $APCC(T) = 37.5\%$, which is exactly shown as the area of the shaded region in Fig. 1(b).

3.2.2. Metrics with combination weights and test costs respectively

We discuss two more complex scenarios, where combination weights and test costs are respectively taken into consideration.

If we take the combination weights into consideration, the contributions of test cases should be the total weighted benefit of them, rather than the number of their covered τ -tuple combinations. Since the weight of τ -tuple combination $c_k \in CombSet$ is denoted as $Weight(c_k) = cw_k$, the total weighted benefit of all executed test cases $T_1, T_2, \dots, T_i$ is

$$AddtlBenefit_i = \sum_{c_k \in \bigcup_{j=1}^i tCov(T_j)} cw_k, \quad i = 1, 2, \dots, m. \quad (14)$$

By utilizing the term $AddtlBenefit_i$ to replace the $AddtlCov_i$ mentioned previously, we get the $APCC_w$ metric for the scenario that combination weights are taken into account. It is denoted as:

$$APCC_w(T) = \frac{\sum_{i=1}^{m-1} \sum_{c_k \in \bigcup_{j=1}^i tCov(T_j)} cw_k}{m \times \sum_{c_k \in CombSet} cw_k}. \quad (15)$$

In another aspect, if the costs of test cases are taken into account when prioritizing combinatorial test cases; we should pay attention to each test case's efficiency of covering combinations. So the step function $fun(x)$ that mentioned previously needs to be modified to satisfy the varying costs of different test cases. Since the cost of test case $T_i \in T$ is denoted as $Cost(T_i) = tc_i$, the input of modified version of step function should be a real number that ranges from 0 to $\sum_{i=1}^m tc_i$, which means the consumed costs in testing process. The output is the same as the previous version. We have:

$$fun_c(x) = \begin{cases} 0, & x < tc_1 \\ |AddtlCov_i|, & \sum_{j=1}^i tc_j \leq x < \sum_{j=1}^{i+1} tc_j. \end{cases} \quad (16)$$

By utilizing the term $fun_c(x)$ to replace the $fun(x)$ used previously, we get the $APCC_c$ metric for the scenario where test costs are taken into account. It is denoted as:

$$APCC_c(T) = \frac{\sum_{i=1}^{m-1} (tc_{i+1} \times |\bigcup_{j=1}^i tCov(T_j)|)}{(\sum_{j=1}^m tc_j) \times |CombSet|}. \quad (17)$$

Take the test suite in Table 3 as an example. There are $APCC_w(T) = 47.22\%$ for order 1-2-3-4, and $APCC_c(T) = 50\%$ for order 4-3-2-1. They are equal to the areas of the shaded regions in Figs. 2(a) and 2(b) and they are also the best results for above two scenarios.

3.2.3. Metric with both combination weights and test costs

Finally, the metric for the scenario, where both the weights of combinations and the costs of test cases are taken into account, can be deduced from the $APCC$, $APCC_w$ and $APCC_c$. For example, it is easy to modify $APCC_c$ by replacing the term $AddtlCov_i$ with another term $AddtlBenefit_i$. Then we have the metric $APCC_{c-w}$ as:

$$APCC_{c-w}(T) = \frac{\sum_{i=1}^{m-1} (tc_{i+1} \times \sum_{c_k \in \bigcup_{j=1}^i tCov(T_j)} cw_k)}{(\sum_{j=1}^m tc_j) \times (\sum_{c_k \in CombSet} cw_k)}. \quad (18)$$

Similarly with $APCC$, $APCC_c$ and $APCC_w$, the metric $APCC_{c-w}$ has the value range $(0, 1)$. The reasons include: (i) The number of test cases in a combinatorial test suite is larger than 1, and $\exists i \in \{1, 2, \dots, m-1\}$ to be $|AddtlCov_i| > 0$. So there must be $APCC_{c-w}(T) > 0$. (ii) In combinatorial test suite, it is impossible to find one test case covering all τ -tuple combinations in $CombSet$. So there must be $|AddtlCov_i| < |CombSet|$, and $APCC_{c-w}(T) < 1$.

To illustrate the $APCC_{c-w}$ metric vividly, we still take the test suite in Table 3 as an example. The weighted benefit and cost of each test case have been listed in the table. If the order of prioritized test cases is 4-1-3-2, there is $APCC_{c-w}(T) = 44.44\%$

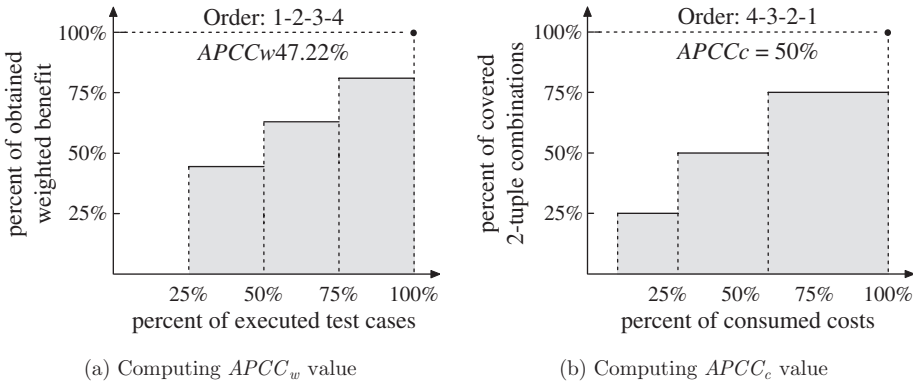


Fig. 2. $APCC_w$ and $APCC_c$ metrics for prioritized test cases.

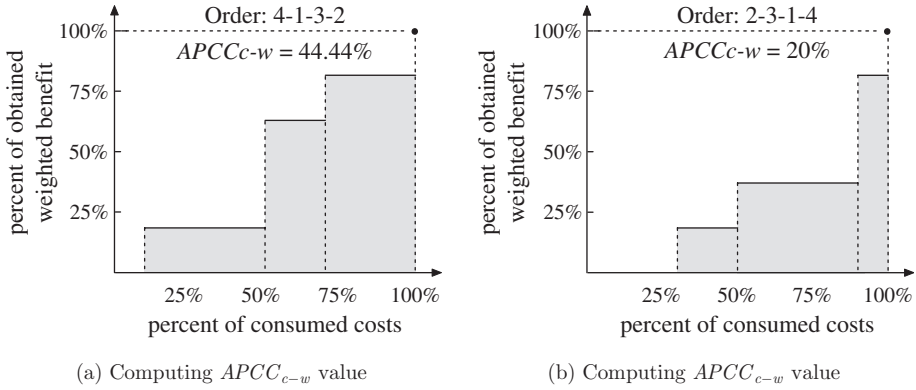


Fig. 3. Example of $APCC_{c-w}$ metric for prioritized test cases.

(see the area of the shaded region in Fig. 3(a)); and if the order of prioritized test cases is 2-3-1-4, there is $APCC_{c-w}(T) = 20\%$ (see the area of the shaded region in Fig. 3(b)). In the scenario where both combination weights and test costs are taken into account, the above two results are respectively the best and worst orders of test cases in Table 3.

3.3. Properties of metrics

Some properties of the proposed $APCC$ metrics (also including $APCC_w$, $APCC_c$ and $APCC_{c-w}$) are discussed in the following sub-section.

Property 1. The $APCC$, $APCC_w$, and $APCC_c$ could be considered as special cases of $APCC_{c-w}$ in the scenario where both combination weights and test costs are ignored, the scenario where only the weights of combinations are taken into account, and the scenario where only the costs of test cases are taken into account.

The above property is clearly illustrated in the process of deducing four metrics. For example, for the scenario where both combination weights and test costs are ignored, we assume there are $Weight(c_k) = 1$ for all τ -tuple combinations and $Cost(test) = 1$ for all test cases, then the $APCC_{c-w}$ metric will degenerate into $APCC$. And for the scenario where only the weights of combinations are taken into account, we assume there are $Cost(test) = 1$ for all test cases, then the $APCC_{c-w}$ metric will degenerate into $APCC_w$ and it is similar for the $APCC_c$. So we could use the general term **$APCC$ metrics** to instead concrete names of four metrics.

Property 2. When $APCC$ metrics (also including $APCC_w$, $APCC_c$ and $APCC_{c-w}$) are used in comparing different test suites' efficiency of combinatorial coverage, only the test suites with the same total costs are comparable.

A counter-example could be given to illustrate the above property. For the simple test object with $F = \{2^3\}$, besides a prioritized test suite T with an order 1-2-3-4 of

test cases shown in Table 3, we can find another prioritized pair-wise test suite with more test cases $T' = \{(0, 0, 0), (1, 1, 1), (0, 1, 0), (1, 0, 0), (0, 0, 1)\}$. By computing $APCC$ metric for T and T' respectively, we get $APCC(T) = 37.5\%$ and $APCC(T') = 45\%$. Although T' has a larger metric value than T , it is evident that T is better than T' since T requires less test cases than T' to achieve the goal of pair-wise testing covering all possible 2-tuple combinations. Other metrics including $APCC_w$, $APCC_c$ and $APCC_{c-w}$ are similar.

Theorem 1. *Let test cost means its time consumed, and the variable t is the time when testing process is terminated exceptionally. If $t \sim U[0, \sum_{j=1}^m tc_j]$, then the value of $APCC_{c-w}$ metric is exactly equal to the mathematical expectation of the percentage of obtained benefit before the exceptional termination.*

Proof. Note that $t \sim U[0, \sum_{j=1}^m tc_j]$, it means the probability of the event that testing process is terminated exceptionally during the period when the i th test case $T_i \in T$ is still running is $p(i) = tc_i / \sum_{j=1}^m tc_j$. Since all executed test cases include $T_1, T_2, \dots$, and T_{i-1} during such period, all covered τ -tuple combinations exactly form the set $AddtlCov_{i-1}$. Therefore, the proportion of obtained benefit in comparison with the total benefit is:

$$g(i) = \frac{\sum_{c_k \in AddtlCov_{i-1}} cw_k}{\sum_{c_k \in CombSet} cw_k} = \frac{\sum_{c_k \in \bigcup_{j=1}^{i-1} tCov(T_j)} cw_k}{\sum_{c_k \in CombSet} cw_k}. \quad (19)$$

So the mathematical expectation (ME) of the proportion of obtained benefit before the exceptional termination in comparison with the total benefit is:

$$\begin{aligned} ME &= \sum_{i=1}^m (p(i) \times g(i)) \\ &= \sum_{i=1}^m \left(\frac{tc_i}{\sum_{j=1}^m tc_j} \times \frac{\sum_{c_k \in \bigcup_{j=1}^{i-1} tCov(T_j)} cw_k}{\sum_{c_k \in CombSet} cw_k} \right) \\ &= \frac{\sum_{i=1}^{m-1} (tc_{i+1} \times \sum_{c_k \in \bigcup_{j=1}^i tCov(T_j)} cw_k)}{(\sum_{j=1}^m tc_j) \times (\sum_{c_k \in CombSet} cw_k)} = APCC_{c-w}. \end{aligned} \quad (20)$$

Therefore, we can conclude that the value of the $APCC_{c-w}$ metric is exactly equal to the mathematical expectation of the percentage of obtained benefit before the exceptional termination of testing process. $\square$

The above theorem provides a physical meaning of $APCC_{c-w}$ metric for the applications where test resource is limited: the prioritized test suite with larger $APCC_{c-w}$ metric value could yield more benefit before the exceptional termination of testing process because of the limitation of resource.

As discussed previously, the time of exceptional termination is assumed to follow a uniform distribution. It is possible in other applications that the termination time

may follow some other distributions within the range $[0, \sum_{j=1}^m tc_j]$. Suppose the probability density function is $den(x)$, then the probability of the event that exceptional termination occurs during the period when the i th test case $T_i \in T$ is still running is:

$$p(i) = \int_{\sum_{j=1}^{i-1} tc_j}^{\sum_{j=1}^i tc_j} den(x) dx. \quad (21)$$

Correspondingly, we have the mathematical expectation (ME) of the percentage of obtained benefit before the exceptional termination as:

$$\begin{aligned} ME &= \sum_{i=1}^m (p(i) \times g(i)) \\ &= \sum_{i=1}^m \left(\int_{\sum_{j=1}^{i-1} tc_j}^{\sum_{j=1}^i tc_j} den(x) dx \times \frac{\sum_{c_k \in \bigcup_{j=1}^{i-1} tCov(T_j)} cw_k}{\sum_{c_k \in CombSet} cw_k} \right). \end{aligned} \quad (22)$$

Such a mathematical expectation could be utilized as a physical meaningful metric for the scenario where the distribution of the exceptional termination time and corresponding probability density function $den(x)$ has been known exactly.

4. Combinatorial Test Case Prioritization Algorithms

After defining the metric, the test case prioritization technique should prioritize test cases in the initial non-prioritized combinatorial test suite T_{init} to increase $APCC_{c-w}$ metric value of reordered test suite as much as possible. Here we utilize two heuristic algorithms, which are based on the “total technique” and “additional technique”, respectively.

4.1. Prioritize according to the total benefit per unit cost

Firstly, we describe the algorithm which prioritizes test cases according to each test case’s total benefit per unit cost. The total benefit per unit cost for each test case $test \in T_{init}$ is defined as:

$$TBPUC(test) = \frac{\sum_{c_k \in tCov(test)} cw_k}{Cost(test)}. \quad (23)$$

After calculating total benefit per unit cost for all m test cases in T_{init} , we sort test cases according to their values of total benefit per unit cost in descending order, to obtain a prioritized test suite T . For those test cases with the same priority, we select randomly. Algorithm 1, which adopts the selection sorting as an example, describes all the details of the total technique.

Algorithm 1 Prioritize test cases according to the total benefit per unit cost (*TotalCTPri*)

Input: T_{init} , Weights of Combinations, Costs of Test Cases

Output: Prioritized Test Suite T

1. Initialize a vector $Count[1 \dots m]$
 2. **for** $i:=1$ **to** m //Compute $TBPUC$ for the i th test case in T_{init}
 3. $Count[i]:=TBPUC(T_{init}[i])$
 4. **end for**
 5. **for** $i:=1$ **to** m //Selection sort according to $TBPUC$ in descending
 6. $Max:=0$; $Best:=0$
 7. **for** $k:=i$ **to** m
 8. **if** $(T_{init}[k] \notin T \text{ AND } Count[k]>Max)$ **then**
 9. $\{Max:=Count[k]; Best:=k\}$
 10. **end if**
 11. **end for**
 12. Insert $T_{init}[Best]$ as the i th test case in T
 13. **end for**
-

Here we analyze the time performance of such an algorithm. When calculating the total benefit per unit cost for a test case, the covered combinations should be visited to count their weights. One test case can cover C_n^τ -tuple combinations, which belong to C_n^τ different subsets $CombSet_r$ ($r \in R$) of $CombSet$. By using binary search, the time complexity of visiting these combinations for one test case is $O(\sum_{r \in R} \log(|CombSet_r|)) < O(C_n^\tau \times \log(a^\tau))$. Besides, the average time complexity of selection sort is $O(m^2)$, and that of the quick sort algorithm is $O(m \times \log(m))$. Above all, the average time complexity of algorithm *TotalCTPri* is not worse than $O(m \times (C_n^\tau \times \log(a^\tau) + \log(m)))$, if we utilize quick sort algorithm to replace the selection sort algorithm described in Algorithm 1. Obviously, the worst time complexity is not worse than $O(m \times (C_n^\tau \times \log(a^\tau) + m))$ no matter which sort of algorithms are utilized.

4.2. Prioritize according to the additional benefit per unit cost

Another opinion is that, all the benefits that have been obtained by previous test cases should be ignored when evaluating current test case. So it is reasonable to consider test case's additional benefit per unit cost. Assume that $l(< m)$ test cases have been selected into prioritized test suite T . When selecting the next one, the additional benefit per unit cost for each test case $test \in T_{init}$, which have not been selected into T , is defined as:

$$ABPUC(test) = \frac{\sum_{c_k \in (tCov(test) - tCov(T))} cw_k}{Cost(test)}. \quad (24)$$

Here, $tCov(T)$ contains all the τ -tuple combinations that are covered by all l test cases that have already been selected into prioritized test suite T .

At each time, we calculate the $ABPUC$ for all test cases that have not been selected into the prioritized test suite T , and select one with the highest $ABPUC$. For the test cases that have the same priority, we select randomly. This operation will repeat until all test cases in T_{init} have been selected into T . Algorithm 2 describes the details.

Algorithm 2 Prioritize test cases according to the additional benefit per unit cost (*AddtlCTPri*)

Input: T_{init} , Weights of Combinations, Costs of Test Cases

Output: Prioritized Test Suite T

```

1. for  $i:=1$  to  $m$ 
2.    $Max:=0$ ;  $Best:=0$ 
3.   for  $k:=1$  to  $m$  //Compute  $ABPUC$  for the  $i$ th test case in  $T_{init}$ 
4.     if  $(T_{init}[k] \notin T)$  then
5.        $Count:=ABPUC(T_{init}[k])$ 
6.       if  $(Count > Max)$  then
7.          $\{Max:=Count; Best:=k\}$ 
8.       end if
9.     end if
10.  end for
11.  Insert  $T_{init}[Best]$  as the  $i$ th test case in  $T$ 
12. end for

```

Here we analyze the time performance of such an algorithm. When selecting the i th test case for T , the number of test cases that are needed to calculate the additional benefit per unit cost is $m - i + 1$. It means that the total time for calculation is $\sum_{i=1}^m (m - i + 1) = (m^2 + m)/2$. The time complexity of calculating all $ABPUC$ is $O(\sum_{r \in R} \log(|CombSet_r - tCov(T)|))$, which is equal to $O(\sum_{r \in R} \log(|CombSet_r|)) < O(C_n^\tau \times \log(a^\tau))$. Therefore, we can conclude that the time complexity of algorithm *AddtlCTPri* is $O(m^2 \times C_n^\tau \times \log(a^\tau))$.

4.3. Properties of algorithms

It is evident that the time performance of algorithm *TotalCTPri* is better than that of *AddtlCTPri*. On the contrary, we guess the $APCC_{c-w}$ metric value of prioritized test suite by the *TotalCTPri* may be a little worse than that prioritized by the *AddtlCTPri*. In fact, they may output the same prioritized test suite sometimes.

Theorem 2. Consider an initial non-prioritized τ -way combinatorial test suite T_{init} . If all the τ -tuple combinations in $CombSet$ are included in T_{init} exactly only

once, then algorithm *TotalCTPri* and *AddtlCTPri* may output the same prioritized test suite.

Proof. For each τ -tuple combination $c \in CombSet$, it is included by only one test case in T_{init} . Therefore, by assuming that $l(< m)$ test cases have been selected into prioritized test suite T , when *AddtlCTPri* selects the next one, we have $tCov(test)-tCov(T)=tCov(test)$ for any test case $test \in T_{init}$, which have not been selected into T . It means that there is $ABPUC(test) = TBPUC(test)$. Therefore, *TotalCTPri* and *AddtlCTPri* will select the same test case at each step, and output the same prioritized test suite finally. $\square$

5. Simulation Experiment

In the following section, some experiments are designed to evaluate the efficiency of proposed cost-cognizant combinatorial test case prioritization technique.

5.1. Improvement of prioritized test suites' efficiency of obtaining benefit

In the first experiment, we will evaluate the performance of prioritized combinatorial test suite generated by two algorithms *TotalCTPri* and *AddtlCTPri*. Here we focus on the test suites' efficiency of obtaining weighted benefit, which could be evaluated by the metric $APCC_{c-w}$. In experiment, the random technique, which output a prioritized test suite with a random order, is also developed for comparison.

5.1.1. Setup experiment

Firstly, we generate initial non-prioritized combinatorial test suite T_{init} . We select four test objects, which represent the systems with fixed-level factors and the systems with mixed-level factors, respectively. Three combinatorial test generation algorithms including Orthogonal Array [17], Algebraic Recursive Construction [18] and Deterministic Density Algorithm (DDA) [19], which are classic algorithms belonging to algebraic direct construction, algebraic recursive construction, and greedy heuristic method, respectively, are selected to generate pair-wise test suites. For each test object, we only utilize a best algorithm ("best" means generating the smallest test suite) of all available ones to generate a pair-wise test suite. The detailed relationship between test objects and their corresponding algorithms are shown in Table 4.

Table 4. Test objects and test generation algorithms.

Test objects	Test suite generation algorithms
$S_1: 5^6$	Orthogonal Array
$S_2: 3^{13}$	Algebraic Recursive Construction
$S_3: 2^3 \times 3^3 \times 4^3 \times 5$	Deterministic Density Algorithm
$S_4: 2^4 \times 6^2 \times 7^2 \times 8^2$	Deterministic Density Algorithm

Secondly, setup weights for all τ -tuple combinations, where $\tau = 2$ in this experiment. For simplicity, we use the method described in [16]: the weights in the range $(0, 1]$ are assigned to individual parametric values, and the weights of τ -tuples could be calculated by multiplying the weights of all corresponding parametric values. There are a total of 4 weight distribution styles, which are utilized in [16], to assign weight for individual parametric values. These styles include:

- *Equal (E)* : all values have the same weight 1;
- *Random (R)*: weights are distributed randomly;
- *Half (H)*: half of the weights for each parameter are set to 0.1, and the other half to 0.9;
- *Reciprocal (P)*: all value weights for f_i are $(1/a_i)^2$.

Thirdly, setup the costs for all individual test cases in non-prioritized pair-wise test suite T_{init} . We adopt all 4 cost distribution styles described in [10], to assign test costs. These styles include:

- *Equal (E)*: all test cases have the same cost 1;
- *Random (R)*: costs are distributed randomly;
- *Mozilla (M)*: utilize the cost distribution in application Mozilla (see Table 5);
- *QTB (Q)*: utilize the cost distribution in application QTB (see Table 6).

Finally, there are 16 different combined distributions. For 4 test objects and 16 combined distributions, we run test case prioritization algorithms and record the $APCC_{c-w}$ metric values of prioritized test suites in Figs. 4–7. In these figures, the vertical axis denotes the metric value, and the horizontal axis denotes different combined distributions. For each combined distribution, results of random, total, and additional techniques are listed from left to right, respectively

Table 5. Cost distribution in application mozilla.

Name	Level	Description	Percent
HTML	1	Least	87%
Printing	2	Normal	1%
Smoke tests	3	More	2%
Buster	4	Most	10%

Table 6. Cost distribution in application QTB.

Level	Description	Percent
1	Less	88%
10	More	12%

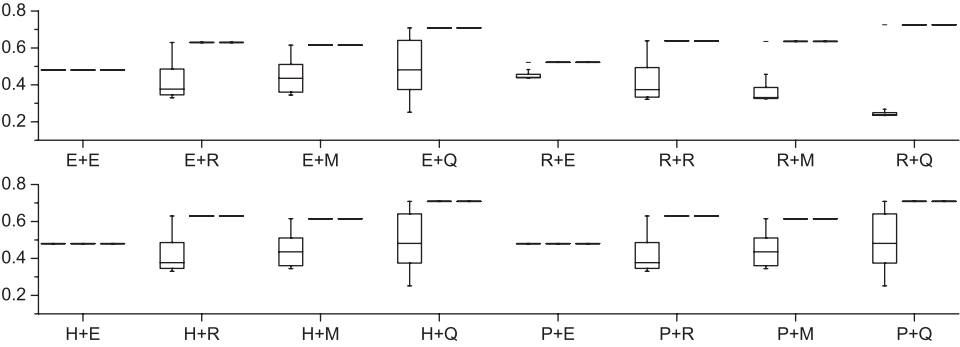


Fig. 4. $APCC_{c-w}$ Metric for different test prioritization techniques ($F = \{5^6\}$, $|T_{init}| = 25$).

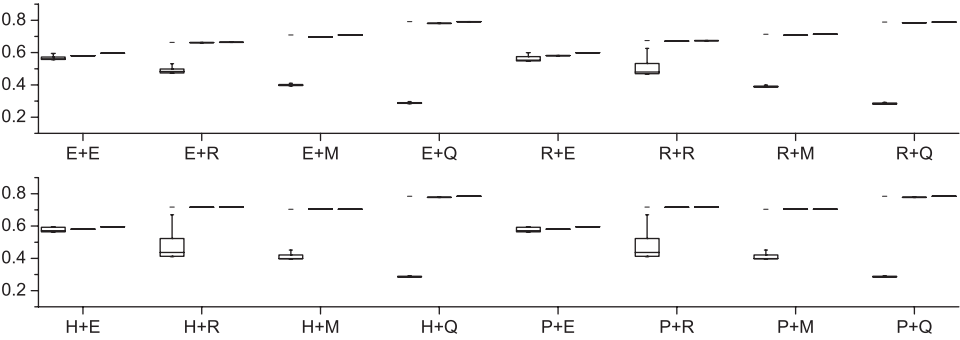


Fig. 5. $APCC_{c-w}$ Metric for different test prioritization techniques ($F = \{3^{13}\}$, $|T_{init}| = 15$).

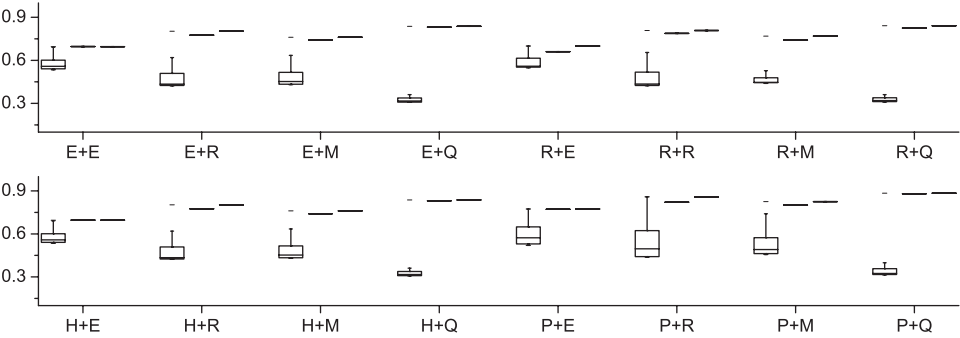


Fig. 6. $APCC_{c-w}$ Metric for different test prioritization techniques ($F = \{2^3 \times 3^3 \times 4^3 \times 5\}$, $|T_{init}| = 25$).

5.1.2. Results and discussion

As displayed in Figs. 4–7, most test suites prioritized by *TotalCTPri* and *AddtlCTPri* have a larger $APCC_{c-w}$ metric value. Only for some special combined distributions, such as the *Equal* weight with *Equal* cost distribution and the

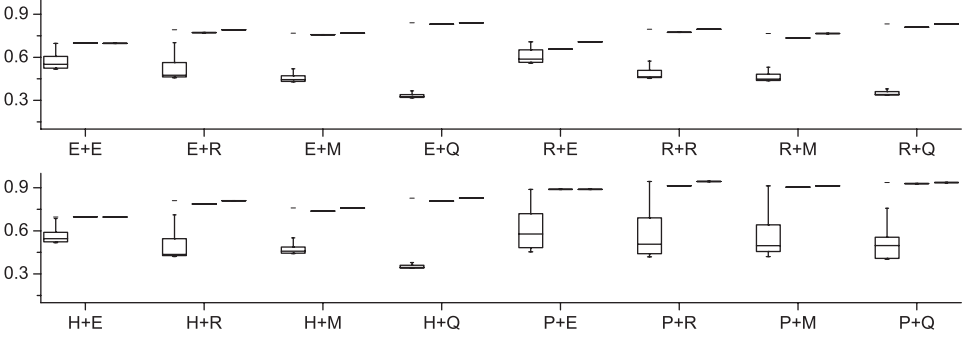


Fig. 7. $APCC_{c-w}$ Metric for different test prioritization techniques ($F = \{2^4 \times 6^2 \times 7^2 \times 8^2\}$, $|T_{init}| = 72$).

Reciprocal weight with *Equal* cost distribution, *TotalCTPri* and *AddtlCTPri* may output the test suites with the same performance to most random results. The possible reason of this phenomenon may include, some weight distributions including *Equal* and *Reciprocal* may lead the situation that all 2-tuple combinations in a given subset $CombSet_r \subseteq CombSet$ have a uniform weight. Therefore, all the test cases will have a uniform *TBPUC* and even *ABPUC*, if the costs of these test cases are set as a uniform value too.

Then compare two proposed algorithms *TotalCTPri* and *AddtlCTPri*. As described in Theorem 2, for all non-prioritized test suites generated from the orthogonal array with $\lambda = 1$, where there is not any redundant 2-tuple combination included in the test suite, two algorithms always generate the prioritized test suites with same $APCC_{c-w}$ metric value. For non-prioritized test suites that generated by other methods including algebraic recursion construction and *DDA*, which does not guarantee the optimality of combinatorial test suites, it is evident that the results of *TotalCTPri* seems a little worse than that of *AddtlCTPri* for most combined distributions. It is also indicated that the gap between the performances of these two algorithms is very narrow. The possible reason for such a narrow gap may be the rare appearance of redundant τ -tuple combinations in a well-designed combinatorial test suite.

It can be concluded from the experimental results that the cost-cognizant combinatorial test case prioritization technique can improve the test suites' efficiency of obtaining testing benefit.

5.2. Compare to existing techniques

In the second experiment, we will compare our proposed technique to a related technique proposed by Bryce, in the scenario where the weights of combination vary. Bryce's technique includes an algorithm named *WDDA*, which can generate prioritized combinatorial test suite directly without initial non-prioritized test suite, for any given weight distributions [16]. Another related technique proposed in [20] will not be taken into comparison, since it can not handle the various weights.

5.2.1. Setup experiment

We still utilize four test objects that shown in Table 4 in the current experiment. Firstly, set weights for parametric values according to four different weight distributions (including *Equal*, *Random*, *Half*, and *Reciprocal*) that have been shown previously, and compute weights for all 2-tuple combinations that need to be covered in test suites. Then, use *WDDA* algorithm to generate prioritized pair-wise test suites T_{WDDA} for different test objects and weight distributions respectively, the sizes of generated test suites are displayed in Table 7.

For comparison, we compare them with four non-prioritized test suites mentioned in the first experiment. The sizes of non-prioritized test suite are also listed in Table 7. Similarly as before, non-prioritized test suites T_{init} will be prioritized by *TotalCTPri* and *AddtlCTPri*, to obtain prioritized test suites T_{Total} and T_{Addtl} , respectively.

Note that for each test object, the sizes of different T_{WDDA} are usually much larger than the size of corresponding non-prioritized test suite (and larger than that of T_{Total} and T_{Addtl} also) for many weight distributions. Since the *APCC* metrics are only comparable in the scenario that test suites with the same summation of test costs, it is necessary to modify some test suites before comparing. If $|T_{init}| < |T_{WDDA}|$, then create totally $|T_{WDDA}| - |T_{init}|$ test cases randomly and insert them into the rear of the order of T_{Total} and T_{Addtl} ; or vice versa.

5.2.2. Results and discussion

By analyzing the comparison results illustrated in Table 7 and Fig. 8, the following conclusions could be obtained.

When algebraic methods including both Orthogonal Array and Algebraic Recursive Construction are available, the prioritized test suites generated by greedy algorithms *WDDA* usually have much more test cases than non-prioritized test suites generated by the algebraic methods. Naturally, after inserting redundant test cases into initial non-prioritized test suite, the final results T_{Total} and T_{Addtl} have greater *APCC_w* metric values than T_{WDDA} almost all the time. So it is evident that our proposed technique has advantage in the scenario where the weights of combinations are taken into consideration in testing.

Table 7. Sizes of prioritized/non-prioritized test suites.

Algorithm	Weight distribution	S_1	S_2	S_3	S_4
WDDA	Equal	26	19	25	72
	Random	31	20	26	73
	Half	29	19	25	72
	Reciprocal	27	20	27	77
Others	—	25 ^a	15 ^b	25 ^c	72 ^c

$S_1: 5^6, S_2: 3^{13}, S_3: 2^3 \times 3^3 \times 4^3 \times 5, S_4: 2^4 \times 6^2 \times 7^2 \times 8^2$ Algorithms:
(a) Orthogonal Array, (b) Recursive Construction, (c) DDA.

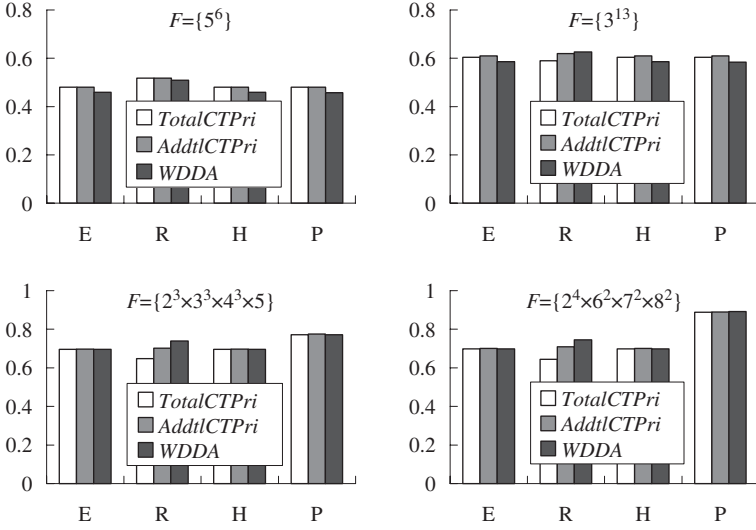


Fig. 8. $APCC_w$ Metric for different test prioritization techniques.

When algebraic methods are not available, we use algorithm *DDA* to generate initial non-prioritized combinatorial test suites. (i) If the weight distribution is set as *Equal*, which means combination weights are ignored, then *WDDA* and *DDA* always generate the same combinatorial test suites. $APCC_w$ metric values of T_{WDDA} , T_{Total} and T_{Addtl} are usually equal. (ii) If the weight distribution is set as *Half*, then the sizes of T_{WDDA} are usually approximate, and even equal, to the sizes of non-prioritized tests suites T_{init} . $APCC_w$ metric values of T_{WDDA} , T_{Total} and T_{Addtl} are very approximate also. (iii) If the weight distribution is *Random* or *Reciprocal*, the sizes of T_{WDDA} are always bigger than that of T_{init} (this phenomenon is also described in [16]). But the $APCC_w$ metric values of T_{WDDA} are not less and even greater than that of T_{Total} and T_{Addtl} .

Above all, it is concluded that the combinatorial test case prioritization technique that proposed in this paper has some advantage in both the sizes and the $APCC$ metric values of prioritized test suites.

5.3. Threats to validity

For some weight distribution style including *Random* and *Half*, there may be numerous different weight distribution instances. The number of these instances is combinatorial explosion. It is similar for some cost distribution styles including *Random*, *Mozilla* and *QTB*. Therefore, except for the combined distribution styles including *Equal + Equal* and *Equal + Reciprocal*, it is impossible to display results for all distribution instances that satisfy one of other 14 combined distribution styles. Actually, we made a reduction in our first experiment described in Sec. 5.1.

For each combined distribution style, we display the results of only one of all possible instances. The lack of exhaustive results may lead to the loss of some possible features of proposed combinatorial test case prioritization technique. The limitations of the second experiment, where we display the results of only one of all possible instances for weight distribution styles *Random* and *Half*, are the same as before.

6. Related Work

People have begun to study combinatorial test case prioritization technique in recent years. All these works can be addressed using two strategies: (i) Pure Prioritization Strategy: prioritize test cases in an existing combinatorial test suite; and (ii) Re-Generation Strategy: incorporate test prioritization into combinatorial test generation [16]. The technique proposed in this paper belongs to the Pure Prioritization Strategy.

For the scenario that neither the importance of combinations nor test costs are taken into consideration, Bryce *et al.* proposed a metric for combinatorial test prioritization, and used the Pure Prioritization Strategy to prioritize combinatorial test cases [20].

The importance of combinations is also taken into account. The covering array involved in the varying combination weights has been named as the biased covering array. Bryce *et al.* adopted the Re-Generation Strategy to generate biased covering array [16]. Their proposed algorithms could handle some application scenarios such as seeding test cases, constraints, etc. Chen and Gu utilized the ant colony algorithm, which belongs to Re-Generation Strategy also, to generate biased covering array [21].

To setup the weights of combinations that evaluate their importance, Qu and Cohen *et al.* proposed the methods based on code coverage and specification [11]. Especially, for the high-configurable systems, they proposed the weighting method based on fault detection and configuration change [22]. In their later works, the cost of installing and building new configurations is also taken into consideration [23].

7. Conclusion

Both combinatorial testing and test case prioritization are practical software testing techniques. Therefore, people have begun to study combinatorial test prioritization techniques. Considering the circumstances that both combination weights and test costs vary, we proposed a novel cost-cognizant combinatorial test case prioritization technique in this paper. Firstly, a series of *APCC* metrics, including *APCC*, *APCC_w*, *APCC_c*, and *APCC_{c-w}* are proposed to assess the performance of prioritized combinatorial test suite. The prioritized test suite with higher *APCC* metric value could yield higher efficiency of covering τ -tuple combination or obtaining testing benefit. So these metrics, which have a physical meaning in the testing process, could be utilized

to guide the findings of prioritized combinatorial test suite. To prioritize combinatorial test cases, two algorithms which are based on well-known total and additional techniques, respectively, are utilized. Computational results suggest that our proposed algorithms could improve combinatorial test suite's efficiency of obtaining weighted benefit. It also indicated that the proposed technique has some advantages over some existing combinatorial test case prioritization techniques.

In our proposed technique, combinatorial test cases are prioritized depending on the combination weights and test costs. In regression testing, which is the scenario where test case prioritization is often applied, the costs of test case could be obtained approximately from the previous running of such test case. So in future works, more techniques that assign weights to parametric values and combinations are needed to assist in the application of combinatorial test prioritization techniques. More empirical studies are also required to assess the power of combinatorial test prioritization techniques. Especially, we need to pay attention to combinatorial test suite's efficiency of fault detection.

Acknowledgments

The work described in this paper was partially supported by the National Natural Science Foundation of China (90818027, 91018005, 60803007, and 61003020), and the Major State Basic Research Development Program of China (973 Program: 2009CB320703).

References

1. Z. Wang, B. Xu, L. Chen and Z. Chen, Cost-effective combinatorial test case prioritization for varying combination weights, in *Proceedings of the 22nd International Conference on Software Engineering and Knowledge Engineering (SEKE2010)*, Redwood City, San Francisco Bay, USA, pp. 273–278.
2. M. Grindal, B. Lindstrom, J. Offutt and S. F. Andler, An evaluation of combination strategies for test case selection. *Empirical Software Engineering*, **11** (2006) 583–611.
3. D. R. Kuhn and D. R. Wallace, Software fault interactions and implications for software testing, *IEEE Transaction on Software Engineering* **30**(6) (2004) 1–4.
4. C. Yilmaz, M. B. Cohen and A. A. Porter, Covering arrays for efficient fault characterization in complex configuration spaces. *IEEE Transaction on Software Engineering*, **32**(1) (2006) 20–34.
5. Z. Wang, B. Yu and C. Nie, Greedy heuristic algorithms to generate variable strength combinatorial test suite, in *Proceedings of the 8th International Conference on Quality Software (QSIC2008)*, Oxford, UK, pp. 155–160.
6. G. Seroussi and N. H. Bshouty, Vector sets for exhaustive testing of logic circuits, *IEEE Transaction on Information Theory* **34**(3) (1988) 513–522.
7. C. Nie and H. Leung, A survey of combinatorial testing, *ACM Computing Surveys* **43**(2) (2011) 11.1–11.29.
8. S. Elbaum, A. G. Malishevsky and G. Rothermel, Prioritizing test cases for regression testing, in *Proceedings of the ACM International Symposium on Software Testing and Analysis (ISSTA2000)*, Portland, Oregon, USA, pp. 102–112.

9. G. Rothermel, R. H. Untch, C. Y. Chu and M. J. Harrold, Prioritizing test cases for regression testing, *IEEE Transactions on Software Engineering* **27**(9) (2001) 929–948.
10. S. Elbaum, A. Malishevsky and G. Rothermel, Incorporating varying test costs and fault severities into test case prioritization, in *Proceedings of the 23rd International Conference on Software Engineering (ICSE2001)*, Toronto, Canada, pp. 329–338.
11. X. Qu, M. B. Cohen and K. M. Woolf, Combinatorial interaction regression testing: A study of test case generation and prioritization, in *Proceedings of the 23th IEEE International Conference on Software Maintenance (ICSM2007)*, Paris, France, pp. 255–264.
12. W. E. Wong, J. R. Horgan, S. London and H. Agrawal, A study of effective regression testing in practice, in *Proceedings of the 8th IEEE International Symposium on Software Reliability Engineering (ISSRE1997)*, Albuquerque, New Mexico, USA, pp. 264–274.
13. A. Srivastava and J. Thiagrajan, Effectively prioritizing tests in development environment, in *Proceedings of the ACM International Symposium on Software Testing and Analysis (ISSTA2002)*, Roma, Italy, pp. 97–106.
14. K. R. Walcott, M. L. Soffa, G. M. Kapfhammer and R. S. Roos, Time-aware test suite prioritization, in *Proceedings of the ACM International Symposium on Software Testing and Analysis (ISSTA2006)*, Portland, Maine, USA, pp. 1–12.
15. D. Jeffrey and N. Gupta, Test case prioritization using relevant slices, in *Proceedings of the 29th International Computer Software and Applications Conference (COMP-SAC2006)*, Chicago, USA, pp. 411–420.
16. R. C. Bryce and C. J. Colbourn, Prioritized interaction testing for pair-wise coverage with seeding and constraints, *Information and Software Technology* **48**(10) (2006) 960–970.
17. R. Mandl, Orthogonal latin squares: An application of experimental design to compiler testing, *Communications of the ACM* **28**(10) (1985) 1054–1058.
18. N. Kobayashi, T. Tsuchiya and K. Tohru, A new method for constructing pair-wise covering designs for software testing, *Information Processing Letters* **81**(2) (2002) 85–91.
19. R. C. Bryce and C. J. Colbourn, The density algorithm for pairwise interaction testing, *Software Testing, Verification and Reliability* **17**(3) (2007) 159–182.
20. R. C. Bryce and A. M. Memon, Test suite prioritization by interaction coverage, in *Proceedings of the Workshop on Domain-Specific Approaches to Software Test Automation (DoSTA2007)*, Dubrovnik, Croatia, pp. 1–7.
21. X. Chen, Q. Gu, X. Zhang and D. Chen, Building prioritized pairwise interaction test suites with ant colony, in *Proceedings of the 9th International Conference on Quality Software (QSIC2009)*, Jeju, Korea, pp. 347–352.
22. X. Qu, M. B. Cohen and G. Rothermel, Configuration-aware regression testing: An empirical study of sampling and prioritization, in *Proceedings of the ACM International Symposium on Software Testing and Analysis (ISSTA2008)*, Seattle, Washington, USA, pp. 75–85.
23. H. Srikanth, M. B. Cohen and X. Qu, Reducing field failures in system configurable software: Cost-based prioritization, in *Proceedings of the 20th IEEE International Symposium on Software Reliability Engineering (ISSRE2009)*, Mysuru, India, pp. 61–70.